

P1729R0

Text Parsing

Published Proposal, 2019-06-14

This version:

<http://wg21.link/P1729R0>

Author:

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper discusses a new text parsing facility to complement the text formatting functionality of [\[P0645\]](#).

Table of Contents

1 Introduction

2 Design

2.1 Format strings

2.2 Safety

2.3 Extensibility

2.4 Locales

2.5 Performance

2.6 Binary footprint

2.7 Integration with chrono

2.8 Impact on existing code

3 Existing work

4 Questions

References

Informative References

§ 1. Introduction

[P0645] has proposed a text formatting facility that provides a safe and extensible alternative to the `printf` family of functions. This paper explores the possibility of adding a symmetric parsing facility which is based on the same design principles and shares many features with [P0645], namely

- [Safety](#)
- [Extensibility](#)
- [Performance](#)
- [Locale control](#)
- [Small binary footprint](#)
- [Integration with chrono](#)

According to [CODESEARCH], a C and C++ codesearch engine based on the ACTCD19 dataset, there are 389,848 calls to `sprintf` and 87,815 calls to `scanf` at the time of writing. So although formatted input functions are less popular than their output counterparts, they are still widely used.

Lack of a general-purpose parsing facility based on format strings has been raised in [P1361] in the context of formatting and parsing of dates and times.

Although having a symmetric parsing facility seems beneficial, not all languages provide it out-of-the-box. For example, Python doesn't have a `scanf` equivalent in the standard library but there is a separate `parse` package ([PARSE]).

Example:

```
std::string key;
int value;
std::scan("answer = 42", "{} = {}", key, value);
//      ~~~~~ ~~~~~ ~~~~~
//      input   format  arguments
//
// Result: key == "answer", value == 42
```

§ 2. Design

The new parsing facility is intended to complement the existing C++ I/O streams library, integrate well with the chrono library, and provide an API similar to `std::format`. This section discusses major features of its design.

§ 2.1. Format strings

As with `printf`, the `scanf` syntax has the advantage of being familiar to many programmers. However, it has similar limitations:

- Many format specifiers like `hh`, `h`, `l`, `j`, etc. are used only to convey type information. They are redundant in type-safe parsing and would unnecessarily complicate specification and parsing.
- There is no standard way to extend the syntax for user-defined types.
- Using `'%'` in a custom format specifier poses difficulties, e.g. for `get_time`-like time parsing.

Therefore we propose a syntax based on [\[PARSE\]](#) and [\[P0645\]](#). This syntax employs `'{'` and `'}'` as replacement field delimiters instead of `'%'`. It will provide the following advantages:

- An easy to parse mini-language focused on the data format rather than conveying the type information
- Extensibility for user-defined types
- Positional arguments
- Support for both locale-specific and locale-independent parsing (see [§2.4 Locales](#))
- Consistency with `std::format` proposed by [\[P0645\]](#).

At the same time most of the specifiers will remain the same as in `scanf` which can simplify, possibly automated, migration.

§ 2.2. Safety

`scanf` is arguably more unsafe than `printf` because `__attribute__((format(scanf, ...)))` ([ATTR]) implemented by GCC and Clang doesn't catch the whole class of buffer overflow bugs, e.g.

```
char s[10];
std::sscanf(input, "%s", s); // s may overflow.
```

Specifying the maximum length in the format string above solves the issue but is error-prone especially since one has to account for the terminating null.

Unlike `scanf`, the proposed facility relies on variadic templates instead of the mechanism provided by `<cstdarg>`. The type information is captured automatically and passed to scanners guaranteeing type safety and making many of the `scanf` specifiers redundant (see §2.1 [Format strings](#)). Memory management is automatic to prevent buffer overflow errors.

§ 2.3. Extensibility

We propose an extension API for user-defined types similar to the one of [P0645]. It separates format string processing and parsing enabling compile-time format string checks and allows extending the format specification language for user types.

The general syntax of a replacement field in a format string is the same as in [P0645]:

```
replacement-field ::= '{' [arg-id] [':' format-spec] '}'
```

where `format-spec` is predefined for built-in types, but can be customized for user-defined types. For example, the syntax can be extended for `get_time`-like date and time formatting

```
auto t = tm();
scan(input, "Date: {0:%Y-%m-%d}", t);
```

by providing a specialization of `scanner` for `tm`:

```

template <>
struct scanner<tm> {
    constexpr scan_parse_context::iterator parse(scan_parse_context& ctx);

    template <class ScanContext>
    typename ScanContext::iterator scan(tm& t, ScanContext& ctx);
};

```

The `scanner<tm>::parse` function parses the `format-spec` portion of the format string corresponding to the current argument and `scanner<tm>::scan` parses the input range `[ctx.begin(), ctx.end())` and stores the result in `t`.

An implementation of `scanner<T>::scan` can potentially use ostream extraction `operator<>>` for user-defined type `T` if available.

§ 2.4. Locales

As pointed out in [\[N4412\]](#):

There are a number of communications protocol frameworks in use that employ text-based representations of data, for example XML and JSON. The text is machine-generated and machine-read and should not depend on or consider the locales at either end.

To address this [\[P0645\]](#) provided control over the use of locales. We propose doing the same for the current facility by performing locale-independent parsing by default and designating separate format specifiers for locale-specific one.

§ 2.5. Performance

The API allows efficient implementation that minimizes virtual function calls and dynamic memory allocations, and avoids unnecessary copies. In particular, since it doesn't need to guarantee the lifetime of the input across multiple function calls, `scan` can take `string_view` avoiding an extra string copy compared to `std::istream`.

We can also avoid unnecessary copies required by `scanf` when parsing string, e.g.

```
std::string_view key;
int value;
std::scan("answer = 42", "{} = {}", key, value);
```

This has lifetime implications similar to returning match objects in [\[P1433\]](#) and iterator or subranges in the ranges library and can be mitigated in the same way.

§ 2.6. Binary footprint

We propose using a type erasure technique to reduce per-call binary code size. The scanning function that uses variadic templates can be implemented as a small inline wrapper around its non-variadic counterpart:

```
string_view::iterator vscan(string_view input, string_view fmt, scan_args a

template <typename... Args>
inline auto scan(string_view input, string_view fmt, const Args&... args) {
    return vscan(input, fmt, make_scan_args(args...));
}
```

As shown in [\[P0645\]](#) this dramatically reduces binary code size which will make `scan` comparable to `scanf` on this metric.

§ 2.7. Integration with chrono

The proposed facility can be integrated with `std::chrono::parse` ([\[P0355\]](#)) via the extension mechanism similarly to integration between chrono and text formatting proposed in [\[P1361\]](#). This will improve consistency between parsing and formatting, make parsing multiple objects easier, and allow avoiding dynamic memory allocations without resolving to deprecated `stringstream`.

Before:

```
std::istringstream is("start = 10:30");
std::string key;
char sep;
std::chrono::seconds time;
is >> key >> sep >> std::chrono::parse("%H:%M", time);
```

After:

```
std::string key;
std::chrono::seconds time;
std::scan("start = 10:30", "{0} = {1:%H:%M}", key, time);
```

Note that the `scan` version additionally validates the separator.

§ 2.8. Impact on existing code

The proposed API is defined in a new header and should have no impact on existing code.

§ 3. Existing work

[\[SCNLIB\]](#) is a C++ library that, among other things, provides a `scan` function similar to the one proposed here. [\[FMT\]](#) has a prototype implementation of the proposal.

§ 4. Questions

Q1: Do we want this?

Q2: API options:

1. Pass arguments by reference and return an iterator:

```
std::string key;
int value;
auto end = std::scan(input, "{} = {}", key, value);
```

This is similar to what `scanf`, istream `operator>>`, and `std::chrono::parse` do.

2. Return an object wrapping an iterator and parsed values:

```
auto result = std::scan<std::string, int>(input, "{} = {}");
auto end = result.end;
std::string key = std::get<0>(result.values);
int value = std::get<1>(result.values);
```

This option is more cumbersome to use because it requires passing all argument types as template arguments to `scan`. It may also require an extra move or copy to extract the argument's value and impose additional requirements on the argument types (at least default constructibility).

Syntactically it can be simplified using structured bindings.

Q3: naming:

1. `scan`
2. `parse`
3. other

The name "parse" is a bit problematic because of ambiguity between format string parsing and input parsing.

Main API	<code>format</code>	<code>scan</code>	<code>parse</code>
Extension point	<code>formatter</code>	<code>scanner</code>	<code>parser</code>
Parse format string	<code>formatter::parse</code>	<code>scanner::parse</code>	<code>parser::parse_format?</code>
Extension function	<code>formatter::format</code>	<code>scanner::scan</code>	<code>parser::parse</code>
Format string parse context	<code>format_parse_context</code>	<code>scan_parse_context</code>	<code>parse_parse_context?</code>
Context	<code>format_context</code>	<code>scan_context</code>	<code>parse_context</code>

§ References

§ Informative References

[ATTR]

Common Function Attributes. URL: <https://gcc.gnu.org/onlinedocs/gcc-8.2.0/gcc/Common-Function-Attributes.html>

[CODESEARCH]

Andrew Tomazos. Code search engine website. URL: <https://codesearch.isocpp.org>

[FMT]

Victor Zverovich et al. The fmt library. URL: <https://github.com/fmtlib/fmt>

[N4412]

Jens Maurer. N4412: Shortcomings of iostreams. URL: <http://open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4412.html>

[P0355]

Howard E. Hinnant; Tomasz Kamiński. Extending <chrono> to Calendars and Time Zones. URL: <https://wg21.link/p0355>

[P0645]

Victor Zverovich. Text Formatting. URL: <https://wg21.link/p0645>

[P1361]

Victor Zverovich; Daniela Engert; Howard E. Hinnant. Integration of chrono with text formatting. URL: <https://wg21.link/p1361>

[P1433]

Hana Dusíková. Compile Time Regular Expressions. URL: <https://wg21.link/p1433>

[PARSE]

Python `parse` package. URL: <https://pypi.org/project/parse/>

[SCNLIB]

Elias Kosunen. scnlib: scanf for modern C++. URL: <https://github.com/eliaskosunen/scnlib>